Tài liệu mô tả mã nguồn

Ứng dụng nhắn tin ngang hàng (P2P)
trong mạng LAN

Tổ chức dự án và cơ chế hoạt động của từng hàm

Các tệp nguồn:	`const_val.py`, `peer.py`, `broadcast.py`, `chatting.py`, `main.py`
Khám phá peer:	UDP broadcast (cổng 9191)
Trò chuyện:	TCP trực tiếp giữa hai peer
Trọng tâm:	`chatting.py` (lớp `ChatManager`)

Tài liệu chỉ mô tả mã nguồn hiện có; không bao gồm đề xuất chỉnh sửa.

Mục lục

1 Tổng quan **3**
1.1 Mục tiêu và ý tưởng chung . . . 3
1.2 Tham số dòng lệnh . . . 3
1.3 Các lệnh trong chương trình . . . 4

2 Tổ chức dự án **4**
2.1 Các tệp và vai trò . . . 4
2.2 Quan hệ giữa các thành phần . . . 4
2.3 Mô hình đa luồng . . . 5
2.4 Dữ liệu dùng chung và các khóa . . . 6

3 Định dạng gói tin **7**
3.1 Gói UDP thông báo hiện diện . . . 7
3.2 Khung tin nhắn TCP . . . 7

4 `const_val.py` **7**

5 `peer.py` **8**
5.1 `class Peer` . . . 8
5.2 `print_help_message()` . . . 9

6 `broadcast.py` **9**
6.1 `Broadcaster.__init__(self, host, bcast, tcp_port)` . . . 9
6.2 `Broadcaster.make_udp_socket(self)` . . . 10
6.3 `Broadcaster.recieve_message(self, host)` . . . 10
6.4 `Broadcaster.broadcast_message(self, host, bcast, tcp_port)` 11
6.5 Ba hàm điều khiển . . . 12

7 `chatting.py` — chi tiết lớp `ChatManager` **12**
7.1 Thuộc tính của lớp . . . 13
7.2 Bảng quan hệ gọi hàm . . . 13
7.3 `ChatManager.__init__(self, tcp_port, host)` . . . 13
7.4 `ChatManager.accept(self)` . . . 14
7.5 `ChatManager.register_connection(self, peer_name, conn)` . . . 15
7.6 `ChatManager.connect_to_peer(self, peer_name)` . . . 16
7.7 `ChatManager.close_connection(self, peer_name)` . . . 17
7.8 `ChatManager.start_chat(self, peer_name)` . . . 18
7.9 `ChatManager.leave_chat(self)` . . . 19

7.10 ChatManager.recv_exact(self, conn, size) 19
7.11 ChatManager.recv_message(self, conn) 20
7.12 ChatManager.send_message(self, peer_name, message) 21
7.13 ChatManager.recieve_loop(self, peer_name, conn) 21
7.14 Vòng đời của một kết nối 23

8 main.py **24**
8.1 Phân tích tham số 24
8.2 Trình tự khởi tạo 24
8.3 Vòng lặp lệnh chính 25
8.4 Xử lý ngoại lệ và thoát 26

9 Kịch bản hoạt động từ đầu đến cuối **27**

A Bảng tổng hợp toàn bộ hàm **28**

1. Tổng quan

1.1 Mục tiêu và ý tưởng chung

Dự án là một chương trình dòng lệnh cho phép nhiều người dùng trong cùng một mạng LAN nhắn tin trực tiếp với nhau mà *không cần máy chủ trung gian*. Mỗi tiến trình chạy chương trình đóng vai trò một *peer* có tên riêng (tham số `--name`). Hoạt động được chia thành hai giai đoạn độc lập:

- **Khám phá (discovery) bằng UDP broadcast.** Cứ mỗi 3 giây, mỗi peer phát một gói UDP tới địa chỉ broadcast, cổng 9191, chứa tên, cổng TCP và trạng thái của mình. Đồng thời mỗi peer lắng nghe cổng 9191 để lập danh sách các peer khác cùng với địa chỉ IP, cổng TCP, trạng thái và thời điểm nhìn thấy gần nhất.
- **Trò chuyện bằng TCP.** Khi người dùng gõ `/chat <tên>`, chương trình mở (hoặc tái sử dụng) một kết nối TCP tới IP và cổng TCP đã học được từ gói UDP. Tin nhắn được đóng khung bằng một header dài cố định 1024 byte chứa độ dài nội dung, theo sau là nội dung văn bản mã hóa UTF-8.

Tin nhắn đến khi người dùng *không* ở trong cuộc trò chuyện với người gửi được cất vào hàng đợi "chưa đọc" và chỉ được in ra khi người dùng mở chat với người đó.

1.2 Tham số dòng lệnh

Tham số	Bắt buộc	Mặc định	Ý nghĩa
`--name`	Có	—	Tên của peer; được các peer khác dùng làm khóa trong danh sách `peers`.
`--bcast`	Không	`255.255.255.255`	Địa chỉ broadcast đích của gói UDP.
`--tcp-port`	Không	`5050`	Cổng TCP lắng nghe; đồng thời được quảng bá trong gói UDP (kiểu `int`).

Ví dụ chạy hai peer trên cùng một máy:

```
python main.py --name Alice --tcp-port 5050
python main.py --name Bob   --tcp-port 5051
```

1.3 Các lệnh trong chương trình

Lệnh	Mô tả	Nơi xử lý
`/list`	Liệt kê các peer đã phát hiện	`main.py`
`/chat <tên>`	Mở cuộc trò chuyện	`main.py` → `ChatManager.start_chat`
<văn bản>	Gửi tin nhắn tới peer đang chat	`main.py` → `ChatManager.send_message`
`/leave`	Rời cuộc trò chuyện hiện tại	`main.py` → `ChatManager.leave_chat`
`/status <online|away>`	Đổi trạng thái của chính mình	`main.py` (ghi `host.state`)
`/help`	Hiển thị trợ giúp	`peer.print_help_message`
`/quit`	Thoát chương trình	`main.py` → `Broadcaster.end_process`
`/send <file>`	Có trong trợ giúp	Chưa có nhánh xử lý trong `main.py`

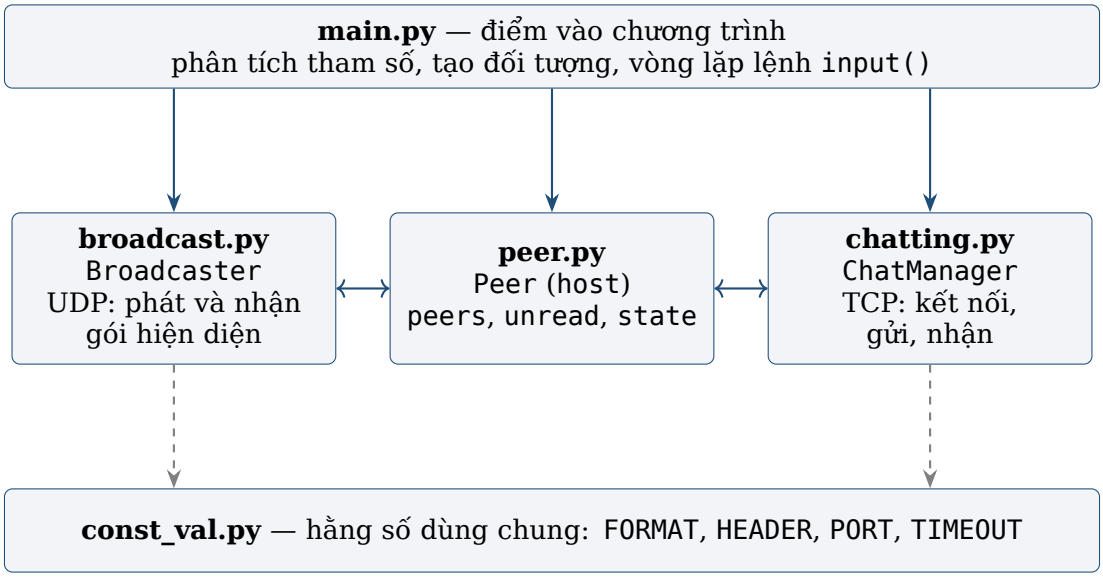
2. Tổ chức dự án

2.1 Các tệp và vai trò

Tệp	Thành phần chính	Vai trò
`const_val.py`	4 hằng số	Cấu hình dùng chung: bảng mã, kích thước header, cổng UDP, ngưỡng timeout.
`peer.py`	Lớp `Peer`; hàm `print_help_message`	Chứa trạng thái dùng chung giữa các luồng (danh sách peer, tin chưa đọc, trạng thái cá nhân) và in trợ giúp.
`broadcast.py`	Lớp `Broadcaster`	Tầng UDP: định kỳ phát gói hiện diện và nhận gói của peer khác.
`chatting.py`	Lớp `ChatManager`	Tầng TCP: lắng nghe kết nối đến, kết nối đi, đóng khung, gửi và nhận tin nhắn.
`main.py`	Khối `if __name__ == "__main__"`	Điểm vào: phân tích tham số, tạo các đối tượng, vòng lặp nhập lệnh.

2.2 Quan hệ giữa các thành phần

Hình 1 thể hiện cách các thành phần liên kết với nhau. Đối tượng `Peer` (biến `host`) là điểm trung tâm: `Broadcaster` ghi danh sách peer vào đó, còn `ChatManager` đọc danh sách ấy để tra cứu địa chỉ và lưu tin chưa đọc vào đó.



Hình 1: Các thành phần của chương trình. Mũi tên liền: tạo đối tượng / đọc–ghi dữ liệu dùng chung; mũi tên đứt: sử dụng hằng số.

Bảng các câu lệnh `import` của từng tệp:

Tệp	Import	Ghi chú
`const_val.py`	(không import gì)	—
`peer.py`	`threading as thd`	Dùng cho `Lock`.
`broadcast.py`	`const_val as c`, `peer as pr`, `threading as thd`, `time as tm`, `socket as sk`	`pr` không được dùng trong nội dung tệp.
`chatting.py`	`const_val as c`, `socket as sk`, `threading as thd`	Nhận đối tượng `host` qua tham số, không import `peer`.
`main.py`	`from broadcast import *`, `peer as p`, `const_val as c`, `chatting as cht`, `threading as thd`, `time as tm`, `socket as sk`, `argparse as ap`	`thd` và `sk` không được dùng.

2.3 Mô hình đa luồng

Chương trình dùng nhiều luồng chạy song song. Tất cả luồng phụ đều là *daemon* (`daemon=True`), nghĩa là chúng bị hủy ngay khi luồng chính kết thúc.

Luồng	Khởi tạo tại	Hàm đích	Nhiệm vụ
Luồng chính	Khối `__main__`	—	Vòng lặp `input()`, xử lý lệnh, gửi tin.
UDP nhận	`Broadcaster.__init__` (chạy bởi `start_listening`)	`recieve_message`	Cập nhật `host.peers`.
UDP phát	`Broadcaster.__init__` (chạy bởi `start_broadcasting`)	`broadcast_message`	Phát gói hiện diện mỗi 3 giây.
TCP chấp nhận	`ChatManager.__init__`	`accept`	Nhận kết nối TCP đến.
TCP nhận (mỗi kết nối 1 luồng)	`register_connection`	`recieve_loop`	Đọc tin nhắn từ một peer.

Tổng số luồng tại một thời điểm là $1 + 3 + N$, với N là số kết nối TCP đang có luồng nhận.

2.4 Dữ liệu dùng chung và các khóa

Dữ liệu	Thuộc về	Cơ chế bảo vệ	Ai truy cập
`host.peers`	`Peer`	`host.peers_lock`	Ghi: `recieve_message`. Đọc: `/list`, `accept`, `connect_to_peer`.
`host.unread`	`Peer`	`host.unread_lock`	Thêm: `recieve_loop`. Lấy ra: `start_chat`.
`host.state`	`Peer`	Không có khóa	Ghi: `/status`. Đọc: `broadcast_message`.
`connections`	`ChatManager`	`self.lock`	`register_connection`, `connect_to_peer`, `close_connection`, `send_message`, `recieve_loop`.
`active_chat`	`ChatManager`	`self.lock`	Ghi: `start_chat`, `leave_chat`. Đọc: `recieve_loop`.
`stop_flag`	`Broadcaster`	`threading.Event`	Đặt: `end_process`. Đọc: hai luồng UDP.
`stop_event`	`ChatManager`	`threading.Event`	Đọc: `accept`, `recieve_loop`. Không có hàm nào gọi `set()`.

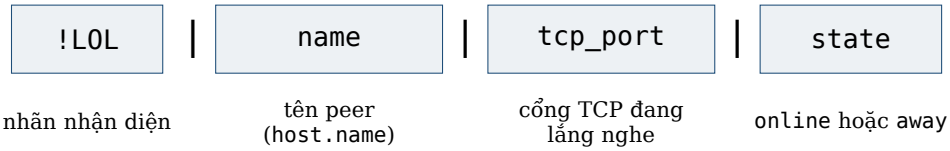
Lưu ý về hành vi

Trong toàn bộ mã nguồn chỉ có *một* nơi giữ đồng thời hai khóa: `recieve_loop` lấy `self.lock` rồi lấy `host.unread_lock` bên trong. Ở mọi nơi khác, các khóa được lấy và nhả riêng rẽ, nên không có chỗ nào lấy hai khóa theo thứ tự ngược nhau.

3. Định dạng gói tin

3.1 Gói UDP thông báo hiện diện

Mỗi gói UDP là một chuỗi văn bản gồm bốn trường nối bằng ký tự `|`, mã hóa UTF-8:

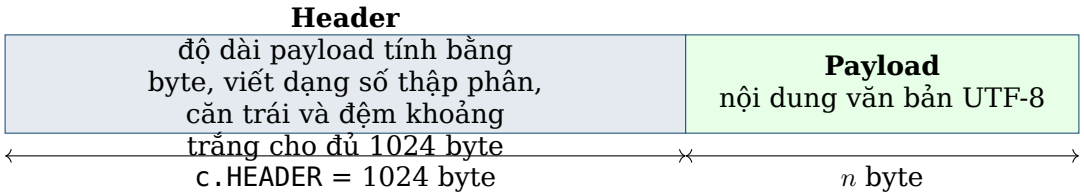


Hình 2: Cấu trúc gói UDP, ví dụ: `!LOL|Alice|5050|online`.

- Gói được gửi tới `(bcast, 9191)` mỗi 3 giây bởi `broadcast_message`.
- Bên nhận đọc tối đa `c.HEADER` = 1024 byte cho mỗi gói (`recvfrom(c.HEADER)`).
- Địa chỉ IP của peer *không* nằm trong nội dung gói; bên nhận lấy IP từ địa chỉ nguồn của datagram (`addr[0]`).

3.2 Khung tin nhắn TCP

TCP là dòng byte không có ranh giới thông điệp, nên mỗi tin nhắn được đóng khung gồm hai phần:



Hình 3: Khung tin nhắn TCP: header cố định 1024 byte, rồi đến payload dài n byte.

- Độ dài ghi trong header là số *byte* sau khi mã hóa, không phải số ký tự. Ví dụ tin nhắn `xin chào` có 8 ký tự nhưng 9 byte (chữ "à" chiếm 2 byte); header sẽ bắt đầu bằng "9" theo sau là 1023 khoảng trắng, và khung dài 1033 byte.
- Bên nhận từ chối (coi như lỗi kết nối) mọi khung có độ dài ngoài khoảng $0 \le n \le$ `max_length`, với `max_length` $= 1024 \times 1024 = 1\,048\,576$ byte.

4. const_val.py

Tệp chỉ chứa bốn hằng số, không có hàm:

```
FORMAT = "utf-8"
HEADER = 1024

PORT = 9191
TIMEOUT = 10
```

Hằng số	Giá trị	Nơi dùng và ý nghĩa
FORMAT	"utf-8"	Bảng mã khi encode/decode mọi dữ liệu: gói UDP, header và payload TCP.
HEADER	1024	Dùng ở hai nơi khác nhau: (1) kích thước header của khung TCP trong send_message và recv_message; (2) kích thước bộ đệm của recvfrom khi nhận gói UDP.
PORT	9191	Cổng UDP: vừa là cổng mà cả hai socket UDP bind, vừa là cổng đích khi phát broadcast.
TIMEOUT	10	Số giây; trong /list, peer có last_seen cũ hơn ngưỡng này được hiển thị là away.

Cổng TCP không nằm trong tệp này vì được truyền qua tham số `--tcp-port`.

5. peer.py

5.1 class Peer

Lớp `Peer` không có hành vi riêng; nó là *kho trạng thái dùng chung* mà các luồng cùng truy cập. Biến `host` trong `main.py` là một thể hiện của lớp này và được truyền cho cả `Broadcaster` lẫn `ChatManager`.

```
class Peer:
    def __init__(self, peer_name):
        self.name = peer_name
        self.state = "online"

        self.peers = {}
        self.peers_lock = thd.Lock()

        self.unread = {}
        self.unread_lock = thd.Lock()
```

`Peer.__init__(self, peer_name)`

Khởi tạo các thuộc tính:

Thuộc tính	Giá trị ban đầu	Ý nghĩa
`name`	`peer_name`	Tên của chính peer này, đặt từ `--name`.
`state`	`"online"`	Trạng thái của chính mình, `"online"` hoặc `"away"`; được quảng bá trong mỗi gói UDP.
`peers`	`\{\}`	Từ điển các peer đã phát hiện (cấu trúc bên dưới).
`peers_lock`	`Lock()`	Bảo vệ `peers`.
`unread`	`\{\}`	Từ điển tên peer → danh sách chuỗi tin nhắn chưa đọc.
`unread_lock`	`Lock()`	Bảo vệ `unread`.

Mỗi phần tử của `peers` có dạng:

```
peers["Bob"] = {
    "ip":        "192.168.1.12",   # lay tu dia chi nguon cua goi UDP
    "port":      5051,             # cong TCP Bob quang ba
    "last_seen": 1730352000.123,   # time.time() luc nhan goi gan nhat
    "state":     "online",         # trang thai Bob quang ba
}
```

5.2 `print_help_message()`

Hàm không tham số, không giá trị trả về. Nó chỉ gọi `print` chín lần: một dòng tiêu đề `[*] Available commands:` và tám dòng mô tả các lệnh `/list`, `/chat`, `<any text>`, `/send`, `/leave`, `/status`, `/help`, `/quit`. Hàm được `main.py` gọi khi người dùng nhập `/help`.

6. `broadcast.py`

Tệp định nghĩa lớp `Broadcaster`, phụ trách toàn bộ phần UDP: phát gói hiện diện của chính mình và thu thập gói của các peer khác vào `host.peers`.

6.1 `Broadcaster.__init__(self, host, bcast, tcp_port)`

```
def __init__(self, host, bcast, tcp_port):
    self.recieve_socket = self.make_udp_socket()
    self.broadcast_socket = self.make_udp_socket()

    self.recieve_thread = thd.Thread(target=self.recieve_message, args=(host,))
    self.recieve_thread.daemon = True

    self.broadcast_thread = thd.Thread(
        target=self.broadcast_message, args=(host, bcast, tcp_port)
    )
    self.broadcast_thread.daemon = True

    self.stop_flag = thd.Event()
```

1. Tạo hai socket UDP riêng bằng `make_udp_socket()`: `recieve_socket` (để nhận) và `broadcast_socket` (để gửi).
2. Tạo luồng `recieve_thread` với hàm đích `recieve_message` và đối số `host`.
3. Tạo luồng `broadcast_thread` với hàm đích `broadcast_message` và đối số `(host, bcast, tcp_port)`.
4. Đặt cả hai luồng ở chế độ daemon.
5. Tạo cờ `stop_flag` (`threading.Event`) để yêu cầu dừng.

Hai luồng chỉ được *tạo* ở bước này, chưa chạy; chúng bắt đầu khi `main.py` gọi `start_listening()` và `start_broadcasting()`.

6.2 `Broadcaster.make_udp_socket(self)`

```
def make_udp_socket(self):
    conn = sk.socket(sk.AF_INET, sk.SOCK_DGRAM)
    conn.setsockopt(sk.SOL_SOCKET, sk.SO_REUSEADDR, True)
    conn.setsockopt(sk.SOL_SOCKET, sk.SO_BROADCAST, True)
    conn.bind(("", c.PORT))
    return conn
```

Hàm tạo và cấu hình một socket UDP, trả về socket đó.

1. `socket(AF_INET, SOCK_DGRAM)`: socket UDP trên IPv4.
2. `SO_REUSEADDR = True`: cho phép nhiều socket cùng `bind` một cặp địa chỉ/cổng. Điều này cần thiết vì *cả hai* socket của `Broadcaster` (và các tiến trình peer khác trên cùng máy) đều gắn vào cổng 9191.
3. `SO_BROADCAST = True`: cho phép socket gửi tới địa chỉ broadcast.
4. `bind(("", c.PORT))`: gắn socket vào mọi giao diện mạng của máy ở cổng UDP 9191 (chuỗi rỗng nghĩa là mọi địa chỉ cục bộ).

6.3 `Broadcaster.recieve_message(self, host)`

Hàm chạy trong luồng `recieve_thread`; nhận gói UDP và cập nhật `host.peers`.

```
def recieve_message(self, host):
    print(f"[*] Broadcasting presence on UDP port {c.PORT}")
    while True:
        data, addr = self.recieve_socket.recvfrom(c.HEADER)
        msg = data.decode(c.FORMAT, errors="replace")
        info = msg.split("|")
        if len(info) == 4 and info[0] == "!LOL" and info[1] == host.name:
            continue

        peer_name = info[1]
        tcp_port = int(info[2])
        peer_ip = addr[0]
        peer_state = info[3]

        with host.peers_lock:
            host.peers[peer_name] = {
                "ip": peer_ip,
                "port": tcp_port,
                "last_seen": tm.time(),
```

```
                "state": peer_state,
            }

        if self.stop_flag.is_set():
            break
```

1. In ra dòng `[*] Broadcasting presence on UDP port 9191`.
2. Vào vòng lặp `while True`.
3. `recvfrom(c.HEADER)` chặn (blocking) cho tới khi có một gói; trả về `data` (bytes, tối đa 1024 byte) và `addr` (cặp IP, cổng nguồn).
4. Giải mã `data` bằng UTF-8; byte không hợp lệ được thay bằng ký tự thay thế (`errors="replace"`) nên bước này không ném ngoại lệ.
5. Tách chuỗi theo `|` thành danh sách `info`.
6. **Bỏ qua gói của chính mình**: nếu `info` có đúng 4 phần tử, phần tử đầu là `"!LOL"` và phần tử thứ hai bằng `host.name` thì `continue`.
7. Trích các trường: `peer_name = info[1]`, `tcp_port = int(info[2])`, `peer_ip = addr[0]`, `peer_state = info[3]`.
8. Trong khối `with host.peers_lock`, ghi đè mục `host.peers[peer_name]` bằng một từ điển mới gồm `ip`, `port`, `last_seen` (thời điểm hiện tại) và `state`. Vì mỗi gói đều ghi đè nên `last_seen` luôn phản ánh lần nhận gần nhất.
9. Kiểm tra `stop_flag`; nếu đã được đặt thì `break`, kết thúc luồng.

Lưu ý về hành vi

- Do hai socket cùng gắn cổng 9191 với `SO_REUSEADDR`, gói broadcast do chính peer phát thường cũng quay lại `recieve_socket`; bước 6 lọc chúng dựa trên *tên*. Hai peer đặt trùng `--name` sẽ bỏ qua gói của nhau.
- Chuỗi `"!LOL"` chỉ được so sánh trong điều kiện ở bước 6; các bước sau lấy trực tiếp `info[1]`, `info[2]`, `info[3]` mà không kiểm tra thêm. Hàm không có `try/except`, nên gói thiếu trường hoặc có cổng không phải số nguyên sẽ làm phát sinh `IndexError` hoặc `ValueError` và kết thúc luồng.
- `stop_flag` chỉ được kiểm tra *sau khi* xử lý xong một gói. Vì `recvfrom` là lệnh chặn, luồng chỉ thoát sau khi nhận thêm một gói kể từ lúc cờ được đặt.
- Mục trong `host.peers` không bao giờ bị xóa; trạng thái "`away`" do timeout được suy ra lúc hiển thị trong `/list`.

6.4 `Broadcaster.broadcast_message(self, host, bcast, tcp_port)`

Hàm chạy trong luồng `broadcast_thread`; định kỳ phát gói hiện diện.

```
def broadcast_message(self, host, bcast, tcp_port):
    print(f"[*] Listening on TCP port {tcp_port}")
    while True:
        msg = f"!LOL|{host.name}|{tcp_port}|{host.state}".encode(c.FORMAT)
        addr = (bcast, c.PORT)
        self.broadcast_socket.sendto(msg, addr)
        if self.stop_flag.is_set():
            break

        tm.sleep(3)
```

1. In ra dòng `[*] Listening on TCP port <tcp_port>`.
2. Mỗi vòng lặp dựng chuỗi `!LOL|<name>|<tcp_port>|<state>` rồi mã hóa UTF-8. `host.state` được đọc lại *ở mỗi vòng*.
3. Đích gửi là `(bcast, 9191)`; gói được gửi bằng `sendto` qua `broadcast_socket`.
4. Nếu `stop_flag` đã được đặt thì thoát vòng lặp (ngay sau lần gửi hiện tại).
5. Ngủ 3 giây rồi lặp lại.

Lưu ý về hành vi

Vì `host.state` được đọc lại mỗi 3 giây (không dùng khóa), việc đổi trạng thái bằng `/status` chỉ được các peer khác nhìn thấy sau tối đa khoảng 3 giây. Sau khi `stop_flag` được đặt, luồng kết thúc ở lần kiểm tra kế tiếp, tức tối đa sau thời gian ngủ còn lại.

6.5 Ba hàm điều khiển

Hàm	Tác dụng
`start_listening(self)`	Gọi `recieve_thread.start()`: bắt đầu luồng nhận gói UDP.
`start_broadcasting(self)`	Gọi `broadcast_thread.start()`: bắt đầu luồng phát gói UDP.
`end_process(self)`	Gọi `stop_flag.set()`: yêu cầu hai luồng UDP dừng (xem lưu ý ở trên về thời điểm chúng thực sự dừng).

7. `chatting.py` — chi tiết lớp `ChatManager`

Vai trò của lớp ChatManager

`ChatManager` quản lý toàn bộ phần TCP của chương trình: lắng nghe kết nối đến, chủ động kết nối tới peer khác, đóng khung rồi gửi tin nhắn, đọc tin nhắn đến, và quyết định tin nhắn nào được in ngay (đang chat với người gửi) hay được cất vào hàng đợi chưa đọc. Mỗi peer có tối đa *một* kết nối TCP đang được quản lý, được tra theo tên peer; kết nối này dùng được cho cả hai chiều bất kể bên nào là bên khởi tạo.

7.1 Thuộc tính của lớp

Thuộc tính	Kiểu	Ý nghĩa
`max_length`	`int` (1048576)	Độ dài payload tối đa mà bên nhận chấp nhận.
`tcp_port`	`int`	Cổng TCP lắng nghe.
`host`	`Peer`	Đối tượng trạng thái chung: `peers`, `peers_lock`, `unread`, `unread_lock`.
`connections`	`dict[str, socket]`	Tên peer → socket TCP đang được quản lý.
`active_chat`	`str` hoặc `None`	Tên peer đang trò chuyện; `None` nếu không ở trong chat nào.
`lock`	`threading.Lock`	Bảo vệ `connections` và `active_chat`.
`stop_event`	`threading.Event`	Cờ dừng cho `accept` và `recieve_loop`.
`listen_socket`	`socket`	Socket TCP lắng nghe kết nối đến.
`listener_thread`	`threading.Thread`	Luồng chạy hàm `accept`.

7.2 Bảng quan hệ gọi hàm

Hàm	Gọi tới	Được gọi bởi
`__init__`	(tạo luồng chạy `accept`)	`main.py` khi khởi tạo `ChatManager`
`accept`	`register_connection`	Luồng `listener_thread`
`register_connection`	(tạo luồng chạy `recieve_loop`)	`accept`, `connect_to_peer`
`connect_to_peer`	`register_connection`	`start_chat`
`close_connection`	—	`send_message`
`start_chat`	`connect_to_peer`	`main.py` (lệnh `/chat`)
`leave_chat`	—	`main.py` (lệnh `/leave`)
`recv_exact`	—	`recv_message`
`recv_message`	`recv_exact`	`recieve_loop`
`send_message`	`close_connection`	`main.py` (vòng lặp chat)
`recieve_loop`	`recv_message`	Luồng tạo bởi `register_connection`

7.3 `ChatManager.__init__(self, tcp_port, host)`

Mục đích: khởi tạo trạng thái, mở cổng TCP lắng nghe và bật luồng chấp nhận kết nối.

Tham số: `tcp_port` — cổng TCP để lắng nghe; `host` — đối tượng `Peer` dùng chung.

Giá trị trả về: không (là hàm khởi tạo).

```
def __init__(self, tcp_port, host):
    self.max_length = 1024 * 1024

    self.tcp_port = tcp_port
    self.host = host
    self.connections = {}
    self.active_chat = None

    self.lock = thd.Lock()

    self.stop_event = thd.Event()

    self.listen_socket = sk.socket(sk.AF_INET, sk.SOCK_STREAM)
    self.listen_socket.setsockopt(sk.SOL_SOCKET, sk.SO_REUSEADDR, True)

    self.listen_socket.bind(("", tcp_port))
    self.listen_socket.listen()
    self.listen_socket.settimeout(2)

    self.listener_thread = thd.Thread(target=self.accept, daemon=True)
    self.listener_thread.start()
```

Các bước:

1. `max_length = 1024 * 1024`: đặt giới hạn 1 MiB cho kích thước payload khi nhận.
2. Lưu `tcp_port` và tham chiếu `host`.
3. `connections = \{\}` và `active_chat = None`: chưa có kết nối nào, chưa chat với ai.
4. Tạo `lock` (`Lock`) và `stop_event` (`Event`).
5. Tạo `listen_socket` kiểu `AF_INET`/`SOCK_STREAM` (TCP trên IPv4).
6. `SO_REUSEADDR = True`: cho phép gắn lại cổng ngay cả khi cổng còn ở trạng thái chờ sau lần chạy trước.
7. `bind(("", tcp_port))`: gắn vào mọi giao diện mạng ở cổng TCP đã chọn.
8. `listen()`: chuyển socket sang chế độ lắng nghe (độ dài hàng đợi chờ do hệ thống chọn).
9. `settimeout(2)`: mỗi lần `accept()` chỉ chờ tối đa 2 giây rồi ném `socket.timeout` (xem hàm `accept`).
10. Tạo `listener_thread` (daemon) chạy `self.accept` và gọi `start()` ngay trong hàm khởi tạo; từ đây peer đã nhận được kết nối đến.

Lưu ý về hành vi

Hàm khởi tạo không bắt ngoại lệ: nếu cổng TCP đang bị tiến trình khác chiếm, `bind` ném `OSError` và ngoại lệ này truyền thẳng ra nơi tạo đối tượng.

7.4 ChatManager.accept(self)

Mục đích: chạy trong luồng riêng, chấp nhận kết nối TCP đến, xác định xem kết nối đến từ peer nào, rồi giao cho `register_connection`.

Tham số: không. **Trả về:** không (vòng lặp chỉ dừng khi `stop_event` được đặt).

```
def accept(self):
    while not self.stop_event.is_set():
        try:
            conn, addr = self.listen_socket.accept()
        except sk.timeout:
            continue

        with self.host.peers_lock:
            peers = [
                name
                for name, info in self.host.peers.items()
                if info.get("ip") == addr[0]
            ]

        if len(peers) != 1:
            conn.close()
            continue

        self.register_connection(peers[0], conn)
```

Các bước:

1. Lặp khi `stop_event` chưa được đặt.
2. Gọi `listen_socket.accept()`. Hàm trả về socket kết nối `conn` và địa chỉ `addr = (ip, cổng nguồn)` của bên gọi. Nếu sau 2 giây không có ai kết nối, `socket.timeout` bị bắt và vòng lặp quay lại đầu; nhờ vậy luồng có cơ hội kiểm tra `stop_event` định kỳ thay vì chặn vô hạn.
3. **Nhận diện peer theo IP.** Trong khối `with host.peers_lock`, duyệt `host.peers` và tạo danh sách `peers` gồm tên của mọi peer có trường `"ip"` bằng IP nguồn `addr[0]`.
4. **Kiểm tra duy nhất:** nếu `len(peers) != 1` thì đóng `conn` và bỏ qua. Có hai tình huống dẫn tới đây: không có peer nào ứng với IP (chưa phát hiện qua UDP), hoặc có từ hai peer trở lên dùng cùng IP (không phân biệt được).
5. Trường hợp đúng một peer: gọi `register_connection(peers[0], conn)`.

Lưu ý về hành vi

- Hàm không đọc bất kỳ dữ liệu nào từ `conn` và không có bước bắt tay; danh tính bên gọi được suy ra hoàn toàn từ IP nguồn đối chiếu với bảng `host.peers`. Cổng nguồn `addr[1]` không được dùng.
- Khi nhiều tiến trình peer chạy trên cùng một máy, chúng có cùng IP. Từ góc nhìn của một peer, nếu bảng `host.peers` chứa từ hai peer trở lên có cùng IP thì kết nối đến từ IP đó bị đóng ở bước 4.
- Chỉ `socket.timeout` được bắt; ngoại lệ khác từ `accept()` sẽ kết thúc luồng này.
- Theo cơ chế của thư viện chuẩn Python, socket trả về từ `accept()` ở chế độ blocking (không timeout) dù socket lắng nghe có timeout 2 giây.

7.5 `ChatManager.register_connection(self, peer_name, conn)`

Mục đích: đưa một socket TCP vừa có (đến hoặc đi) vào bảng `connections` dưới tên `peer_name` và bật luồng đọc cho nó.

Tham số: `peer_name` — tên peer ở đầu kia; `conn` — socket TCP đã kết nối. **Trả về:** không.

```
def register_connection(self, peer_name, conn):
    with self.lock:
        if peer_name in self.connections:
            conn.close()
            return

        self.connections[peer_name] = conn

    recv_thread = thd.Thread(
        target=self.recieve_loop, args=(peer_name, conn), daemon=True
    )
    recv_thread.start()
```

Các bước:

1. Lấy `self.lock`.
2. **Chính sách trùng lặp:** nếu đã có mục tên `peer_name`, *đóng socket mới* và thoát (giữ nguyên kết nối cũ). Do đó mỗi peer có tối đa một kết nối.
3. Nếu chưa có, lưu `connections[peer_name] = conn`.
4. Nhả khóa, rồi tạo và khởi động luồng daemon chạy `recieve_loop(peer_name, conn)`.

Lưu ý về hành vi

Việc kiểm tra và ghi nằm trong cùng một khối khóa nên hai luồng không thể cùng đăng ký một tên. Luồng nhận được khởi động *ngoài* khối khóa. Khi socket bị đóng vì trùng lặp, không có luồng nhận nào được tạo cho socket đó.

7.6 `ChatManager.connect_to_peer(self, peer_name)`

Mục đích: bảo đảm có một kết nối TCP tới `peer_name`: tái sử dụng nếu đã có, nếu chưa thì chủ động kết nối.

Tham số: `peer_name`. **Trả về:** `bool` — `True` nếu đến cuối hàm có kết nối khả dụng do chính hàm này đảm bảo, `False` nếu thất bại.

```
def connect_to_peer(self, peer_name):
    with self.lock:
        if peer_name in self.connections:
            return True

    with self.host.peers_lock:
        info = self.host.peers.get(peer_name)

    if info is None:
        print(f"[*] {peer_name} not found.")
        return False

    conn = sk.socket(sk.AF_INET, sk.SOCK_STREAM)
    conn.settimeout(5)

    addr = (info["ip"], info["port"])
    try:
        conn.connect(addr)
    except:
```

```
        conn.close()
        return False

    conn.settimeout(None)
    self.register_connection(peer_name, conn)

    with self.lock:
        return self.connections.get(peer_name) is conn
```

Các bước:

1. **Tái sử dụng:** trong khối `self.lock`, nếu `peer_name` đã có trong `connections` thì trả về `True` ngay.
2. **Tra địa chỉ:** trong khối `host.peers_lock`, lấy `info = host.peers.get(peer_name)`.
3. Nếu `info` là `None` (chưa phát hiện peer này), in `[*] <tên> not found.` và trả về `False`.
4. Tạo socket TCP mới, đặt `settimeout(5)` để việc kết nối chờ tối đa 5 giây.
5. Kết nối tới `(info["ip"], info["port"])` — IP từ gói UDP, cổng là cổng TCP mà peer đã quảng bá. Mọi ngoại lệ (hết giờ, bị từ chối, không tới được mạng…) bị bắt bởi `except:` trần: đóng socket và trả về `False`; ở nhánh này hàm không in thông báo.
6. `settimeout(None)`: đưa socket về chế độ blocking không timeout để luồng nhận có thể chờ dữ liệu vô hạn.
7. Gọi `register_connection(peer_name, conn)`.
8. **Xác nhận:** trong khối `self.lock`, trả về kết quả của biểu thức `connections.get(peer_name) is conn`.

Lưu ý về hành vi

- Bước 8 so sánh bằng `is`, tức kiểm tra mục trong bảng có đúng là socket *vừa tạo* hay không. Nếu trong lúc đang kết nối, một kết nối đến từ cùng peer được `accept` đăng ký trước, `register_connection` sẽ đóng socket mới và bước 8 trả về `False`, dù bảng `connections` đã có một kết nối với peer đó.
- Bước 1 chỉ kiểm tra sự tồn tại của mục, không kiểm tra socket còn sống hay không.
- `info` là tham chiếu tới từ điển trong `host.peers`; `info["ip"]` và `info["port"]` được đọc ở bước 5 bên ngoài khối khóa.

7.7 ChatManager.close_connection(self, peer_name)

Mục đích: gỡ kết nối khỏi bảng và đóng socket.

Tham số: `peer_name`. **Trả về:** không.

```
def close_connection(self, peer_name):
    with self.lock:
        conn = self.connections.pop(peer_name, None)

    if conn:
        conn.close()
```

Các bước:

1. Trong khối `self.lock`, pop mục `peer_name` khỏi `connections`, mặc định `None` nếu không có.
2. Ngoài khối khóa, nếu thực sự lấy được socket thì gọi `conn.close()`.

Việc đóng socket nằm ngoài khối khóa để không giữ khóa trong lúc thực hiện thao tác của hệ điều hành. Hàm an toàn khi gọi cho tên không tồn tại (không làm gì). Trong mã hiện tại chỉ `send_message` gọi hàm này, khi việc gửi thất bại.

7.8 `ChatManager.start_chat(self, peer_name)`

Mục đích: mở cuộc trò chuyện với `peer_name`: bảo đảm có kết nối, đánh dấu peer này là đối tượng đang chat, và in các tin nhắn chưa đọc.

Tham số: `peer_name`. **Trả về:** `True` nếu thành công, `False` nếu không kết nối được.

```
def start_chat(self, peer_name):
    if not self.connect_to_peer(peer_name):
        print(f"[*] Cannot connect to {peer_name}")
        return False

    with self.lock:
        self.active_chat = peer_name

    with self.host.unread_lock:
        messages = self.host.unread.pop(peer_name, [])

        for message in messages:
            print(f"[{peer_name}] {message}")

    return True
```

Các bước:

1. Gọi `connect_to_peer(peer_name)`. Nếu trả về `False`, in `[*] Cannot connect to <tên>` và trả về `False`; `active_chat` không bị thay đổi.
2. Trong khối `self.lock`, đặt `active_chat = peer_name`. Từ lúc này, tin nhắn đến từ peer đó sẽ được in thẳng ra màn hình thay vì cất vào hàng đợi.
3. Trong khối `host.unread_lock`, lấy toàn bộ danh sách chưa đọc của peer đó ra khỏi `host.unread` (`pop`, mặc định danh sách rỗng) — việc `pop` đồng thời xóa hàng đợi.
4. In từng tin theo định dạng `[<tên>] <nội dung>`, theo thứ tự đã nhận.
5. Trả về `True`.

Lưu ý về hành vi

- Hàng đợi chưa đọc nằm trong đối tượng `host` (`Peer`), không nằm trong `ChatManager`.
- Việc đặt `active_chat` (bước 2) và lấy tin chưa đọc (bước 3) nằm trong hai khối khóa riêng biệt. Tin nhắn đến đúng giữa hai bước sẽ thấy `active` là đúng và được in ngay.
- Việc in các tin chưa đọc diễn ra khi đang giữ `host.unread_lock`.
- `start_chat` không bao giờ tạo vòng nhập tin; vòng nhập tin nằm trong

`main.py`.

7.9 ChatManager.leave_chat(self)

Mục đích: thoát cuộc trò chuyện hiện tại. **Tham số:** không. **Trả về:** không.

```
def leave_chat(self):
    with self.lock:
        self.active_chat = None
```

Hàm chỉ đặt `active_chat = None` dưới khóa. Hàm *không* đóng socket và không xóa mục trong `connections`; kết nối TCP và luồng nhận vẫn tiếp tục chạy. Kể từ lúc này, tin nhắn mới từ peer vừa rời được xử lý như tin chưa đọc: lưu vào `host.unread` và in thông báo `[*] New message from <tên> (<n> unread)`.

7.10 ChatManager.recv_exact(self, conn, size)

Mục đích: đọc *đúng* `size` byte từ socket.

Tham số: `conn` — socket; `size` — số byte cần đọc. **Trả về:** `bytes` dài đúng `size`, hoặc `None` nếu đầu kia đã đóng kết nối trước khi đọc đủ.

```
def recv_exact(self, conn, size):
    data = bytearray()

    while len(data) < size:
        chunk = conn.recv(size - len(data))

        if not chunk:
            return None
        data.extend(chunk)

    return bytes(data)
```

Lý do tồn tại: TCP là dòng byte; một lần `recv(n)` có thể trả về *ít hơn* n byte (do dữ liệu đến thành nhiều đoạn). Hàm này lặp cho tới khi gom đủ.

Các bước:

1. Tạo bộ đệm `bytearray` rỗng.
2. Khi chưa đủ `size` byte: gọi `conn.recv(size - len(data))` để xin đúng phần còn thiếu.
3. Nếu `recv` trả về chuỗi rỗng (`b""`), nghĩa là đầu kia đã đóng kết nối có trật tự: trả về `None`.
4. Nối đoạn nhận được vào bộ đệm và lặp lại.
5. Khi đủ, đổi sang `bytes` và trả về.

Ví dụ: với `size = 1024`, nếu `recv` lần lượt trả về 300 và 724 byte thì hàm gom hai đoạn và trả về khối 1024 byte.

Lưu ý về hành vi

Hàm chặn cho đến khi đủ dữ liệu hoặc kết nối đóng. Khi `size = 0`, vòng lặp không chạy và hàm trả về `b""` (không phải `None`). Ngoại lệ của `recv` (ví dụ `ConnectionResetError`) không được bắt ở đây mà truyền lên hàm gọi.

7.11 `ChatManager.recv_message(self, conn)`

Mục đích: đọc và giải mã *một* khung tin nhắn hoàn chỉnh (header + payload).

Tham số: `conn`. **Trả về:** chuỗi `str` là nội dung tin nhắn, hoặc `None`.

```
def recv_message(self, conn):
    header = self.recv_exact(conn, c.HEADER)
    if header is None:
        return None

    length = int(header.decode(c.FORMAT).strip())

    if not (0 <= length <= self.max_length):
        return None

    payload = self.recv_exact(conn, length)
    if payload is None:
        return None

    return payload.decode(c.FORMAT)
```

Các bước:

1. Đọc đúng `c.HEADER` = 1024 byte header bằng `recv_exact`. Nếu `None` (đầu kia đóng), trả về `None`.
2. Giải mã header thành chuỗi, `strip()` bỏ khoảng trắng đệm, rồi `int(...)` để có `length` — số byte của payload.
3. **Kiểm tra giới hạn:** nếu `length` không nằm trong $[0,$ `max_length`] thì trả về `None`.
4. Đọc đúng `length` byte payload bằng `recv_exact`. Nếu `None`, trả về `None`.
5. Giải mã payload bằng UTF-8 và trả về chuỗi.

Lưu ý về hành vi

- Giá trị `None` biểu thị hai nhóm tình huống: đầu kia đóng kết nối (ở header hoặc ở payload) và độ dài ngoài khoảng cho phép. `recieve_loop` xử lý cả hai giống nhau.
- Header không phải số (`int` ném `ValueError`) hoặc payload không phải UTF-8 hợp lệ (`decode` ném `UnicodeDecodeError`) *không* trả về `None` mà ném ngoại lệ lên `recieve_loop`.
- Độ dài 0 là hợp lệ: `recv_exact(conn, 0)` trả về `b""` và hàm trả về chuỗi rỗng.

7.12 ChatManager.send_message(self, peer_name, message)

Mục đích: đóng khung và gửi một tin nhắn tới `peer_name` qua kết nối đã có.

Tham số: `peer_name`; `message` — chuỗi cần gửi. **Trả về:** `True` nếu gửi xong, `False` nếu không có kết nối hoặc gửi lỗi.

```
def send_message(self, peer_name, message):
    with self.lock:
        conn = self.connections.get(peer_name)

    if conn is None:
        return False

    try:
        payload = message.encode(c.FORMAT)
        header = str(len(payload)).ljust(c.HEADER).encode(c.FORMAT)

        conn.sendall(header + payload)

        return True
    except:
        self.close_connection(peer_name)
        return False
```

Các bước:

1. Trong khối `self.lock`, lấy socket `conn = connections.get(peer_name)`.
2. Nếu `conn` là `None` thì trả về `False`. Hàm *không* tự mở kết nối mới.
3. Mã hóa `message` thành `payload` (UTF-8).
4. Dựng header: `str(len(payload))` là độ dài tính bằng *byte*; `.ljust(c.HEADER)` đệm khoảng trắng bên phải cho đủ 1024 ký tự; rồi mã hóa thành bytes (đều là ASCII nên đúng 1024 byte).
5. `conn.sendall(header + payload)`: gửi toàn bộ khung trong một lệnh `sendall`, bảo đảm header và payload đi liền nhau và được gửi hết.
6. Thành công: trả về `True`.
7. Mọi ngoại lệ ở các bước 3–5 bị bắt bởi `except:` trần: gọi `close_connection(peer_name)` để gỡ và đóng kết nối rồi trả về `False`.

Lưu ý về hành vi

- Việc gửi diễn ra ngoài khối khóa (khóa chỉ dùng để lấy tham chiếu socket).
- `send_message` không kiểm tra `max_length` khi gửi; giới hạn chỉ được áp dụng ở phía nhận trong `recv_message`.
- Mỗi tin nhắn tốn ít nhất 1024 byte header, bất kể nội dung ngắn đến đâu.
- Trong `main.py`, giá trị trả về của hàm này không được kiểm tra, nên việc gửi thất bại không kèm thông báo nào trên màn hình.

7.13 ChatManager.recieve_loop(self, peer_name, conn)

Mục đích: vòng lặp đọc liên tục của *một* kết nối; mỗi kết nối có một luồng riêng chạy hàm này. Đây là nơi quyết định tin nhắn được in ngay hay được cất vào hàng đợi chưa đọc.

Tham số: `peer_name` — tên peer ở đầu kia; `conn` — socket của kết nối. **Trả về:** không.

```
def recieve_loop(self, peer_name, conn):
    try:
        while not self.stop_event.is_set():
            message = self.recv_message(conn)

            if message is None:
                break

            with self.lock:
                active = self.active_chat == peer_name

                if not active:
                    with self.host.unread_lock:
                        unread = self.host.unread.setdefault(peer_name, [])
                        unread.append(message)
                        count = len(unread)

            if active:
                print(f"[{peer_name}] {message}")
            else:
                print(f"[*] New message from {peer_name} ({count} unread)")
    except:
        with self.lock:
            if self.connections.get(peer_name) is conn:
                self.connections.pop(peer_name)

        conn.close()
```

Luồng xử lý một tin nhắn

1. Lặp khi `stop_event` chưa được đặt.
2. `message = recv_message(conn)` — chặn cho tới khi nhận trọn một khung.
3. Nếu `message` là `None` (đầu kia đóng hoặc khung không hợp lệ) thì `break`.
4. Vào khối `self.lock` và tính `active = (self.active_chat == peer_name)`, tức người dùng có đang chat với chính peer gửi tin này không.
5. **Nếu không đang chat** (`active` sai): vẫn trong khối `self.lock`, lấy thêm `host.unread_lock`, dùng `setdefault` để lấy (hoặc tạo) danh sách chưa đọc của peer, thêm tin vào cuối danh sách và ghi `count = len(unread)`.
6. Ra khỏi các khối khóa, in kết quả:
   - Đang chat: `[<tên>] <nội dung>`.
   - Không chat: `[*] New message from <tên> (<count> unread)`.
7. Quay lại bước 2.

Vì sao việc thêm vào hàng đợi nằm trong khối `self.lock`

Việc đọc `active_chat` và thêm tin vào `host.unread` được thực hiện như một thao tác không bị xen ngang bởi `start_chat` hoặc `leave_chat` (cũng lấy `self.lock`). Nhờ đó với mỗi tin nhắn chỉ xảy ra một trong hai khả năng: tin được thêm vào hàng đợi *trước* khi `active_chat` đổi (và sau đó được `start_chat` lấy ra in), hoặc

`active_chat` đã đổi nên tin được in thẳng. Tin không bị bỏ sót. Đây cũng là nơi duy nhất lồng hai khóa (`self.lock` → `host.unread_lock`). Các lệnh `print` nằm ngoài khối khóa.

Ba cách kết thúc của hàm

Tình huống	Đường thoát	Hệ quả
`stop_event` được đặt	Điều kiện `while` sai	Hàm kết thúc bình thường. (Trong mã hiện tại không có đoạn nào gọi `stop_event.set()`.)
`recv_message` trả `None` (đầu kia đóng hoặc khung lỗi)	`break`	Hàm kết thúc bình thường; khối `except` *không* chạy, nên socket *không* bị đóng và mục trong `connections` vẫn còn.
Có ngoại lệ (`ConnectionResetError`, `ValueError`, `UnicodeDecodeError`, `OSError`…)	Khối `except:`	Gỡ mục khỏi `connections` *nếu mục đó vẫn là đúng socket này*, rồi đóng `conn`.

Lưu ý về hành vi

- Điều kiện `connections.get(peer_name) is conn` trong khối `except` bảo đảm chỉ gỡ mục khi nó còn trỏ tới đúng socket của luồng này. Ví dụ, nếu `send_message` đã gọi `close_connection` (gỡ mục và đóng socket), luồng nhận có thể gặp ngoại lệ trên socket đã đóng; lúc đó mục không còn là `conn` nên không bị gỡ lần nữa.
- Khi đầu kia đóng kết nối theo cách thông thường, luồng nhận thoát bằng `break` và mục vẫn nằm trong `connections` cho đến khi `send_message` gặp lỗi (khi đó `close_connection` gỡ nó) hoặc có ngoại lệ xảy ra trong `recieve_loop`.
- Biến `count` chỉ được gán khi tin nhắn đi vào hàng đợi, và chỉ được dùng ở nhánh `else` của lệnh in, nên luôn có giá trị khi được đọc.
- Trong lúc người dùng đang chờ ở lời nhắc `input()`, các dòng in từ luồng nhận được xuất ra màn hình mà chương trình không vẽ lại lời nhắc.

7.14 Vòng đời của một kết nối

Bảng tổng hợp ảnh hưởng của từng sự kiện lên `connections` và `active_chat`:

Sự kiện	Hàm xử lý	connections	active_chat
Kết nối đến từ IP không xác định hoặc trùng nhiều peer	accept	Không đổi (socket bị đóng)	Không đổi
Kết nối đến từ peer chưa có kết nối	accept → register_connection	Thêm mục; bật luồng nhận	Không đổi
Đăng ký khi đã có mục cùng tên	register_connection	Giữ mục cũ, đóng socket mới	Không đổi
/chat tới peer chưa kết nối	start_chat → connect_to_peer	Thêm mục; bật luồng nhận	Đặt thành tên peer
/chat tới peer đã có kết nối	start_chat	Tái sử dụng mục cũ	Đặt thành tên peer
/leave	leave_chat	Giữ nguyên	Đặt None
Gửi tin thất bại	send_message → close_connection	Gỡ mục, đóng socket	Không đổi
Đầu kia đóng kết nối	recieve_loop (break)	Mục vẫn còn	Không đổi
Ngoại lệ khi nhận	recieve_loop (except)	Gỡ mục (nếu đúng socket), đóng socket	Không đổi

8. main.py

Toàn bộ mã nằm trong khối `if __name__ == "__main__":`, bọc trong `try ... except KeyboardInterrupt`. Tệp không định nghĩa hàm hay lớp.

8.1 Phân tích tham số

Dùng `argparse` khai báo ba tham số: `--name` (bắt buộc), `--bcast` (mặc định `255.255.255.255`) và `--tcp-port` (kiểu `int`, mặc định `5050`). Kết quả lưu trong biến `lst`.

8.2 Trình tự khởi tạo

1. `host = p.Peer(lst.name)`: tạo kho trạng thái dùng chung.
2. `broadcaster = Broadcaster(host, lst.bcast, lst.tcp_port)`: tạo hai socket UDP và hai luồng (chưa chạy).
3. `broadcaster.start_broadcasting()` rồi `broadcaster.start_listening()`: bắt đầu phát và nhận gói UDP.
4. In `Hello` (dòng in kiểm tra trong mã).
5. `manager = cht.ChatManager(lst.tcp_port, host)`: mở cổng TCP, bật luồng `accept`. Đối tượng này dùng chung `host` với `broadcaster`.
6. In `Hello` lần nữa, rồi in `[*] Type /help for instructions`.

8.3 Vòng lặp lệnh chính

Mỗi vòng đọc `command = input("> ").split()`, tức tách dòng nhập theo khoảng trắng thành danh sách token. Dòng rỗng bị bỏ qua. Việc xử lý dựa trên `command[0]`:

Lệnh	Hành vi
`/quit`	Gọi `broadcaster.end_process()`, in `[*] Connection closed`, thoát vòng lặp. Chương trình kết thúc và các luồng daemon bị hủy cùng tiến trình.
`/help`	Gọi `p.print_help_message()`.
`/list`	Xem bên dưới.
`/status` (đúng 2 token)	Nếu đối số là `online` hoặc `away` thì gán vào `host.state` và in `[*] Your status set to "<trạng thái>"`; ngược lại in thông báo lỗi và `continue`.
`/chat`	Xem bên dưới.

Lệnh `/list`

```
current_time = tm.time()

with host.peers_lock:
    index = 1
    peers_snapshot = host.peers.copy()
    for name, info in peers_snapshot.items():
        current_state = info["state"]

        if current_time - info["last_seen"] > c.TIMEOUT:
            current_state = "away"

        print(f"    {index}. {name} ({info['ip']}) {current_state}")

        index += 1
```

1. Lấy thời điểm hiện tại `current_time`.
2. Trong khối `host.peers_lock`, tạo bản sao nông `peers_snapshot` của `host.peers`.
3. Với mỗi peer, trạng thái hiển thị mặc định là trạng thái peer đã quảng bá; nếu `current_time - last_seen > c.TIMEOUT` (10 giây) thì hiển thị `away`.
4. In dòng dạng `\ \ \ \ 1. Bob (192.168.1.12) online`, với số thứ tự đếm từ 1 theo thứ tự chèn của từ điển.

Toàn bộ vòng in nằm bên trong khối `with host.peers_lock`.

Lệnh `/chat` và chế độ chat

```
elif command[0] == "/chat":
    if len(command) != 2:
        print("[*] /chat syntax must be /chat <peer_name>")
        continue
```

```
    peer_name = command[1]

    manager.start_chat(peer_name)

    while True:
        message = input("> ").strip()

        if not message:
            continue

        if message == "/leave":
            print(f"[*] You have left the chat with {peer_name}")
            manager.leave_chat()
            break

        manager.send_message(peer_name, message)
```

1. Nếu số token khác 2, in nhắc cú pháp và quay lại vòng lặp lệnh.
2. Gán `peer_name = command[1]` và gọi `manager.start_chat(peer_name)` (kết nối, đặt `active_chat`, in tin chưa đọc).
3. Vào *vòng lặp chat* lồng bên trong: mỗi dòng nhập được `strip()` (giữ nguyên các khoảng trắng ở giữa, khác với `.split()` ở vòng lệnh chính).
4. Dòng rỗng bị bỏ qua. Dòng đúng bằng `/leave` thì in thông báo rời chat, gọi `manager.leave_chat()` và thoát vòng chat, quay về vòng lệnh chính.
5. Mọi dòng khác được gửi nguyên văn bằng `manager.send_message(peer_name, message)`.

8.4 Xử lý ngoại lệ và thoát

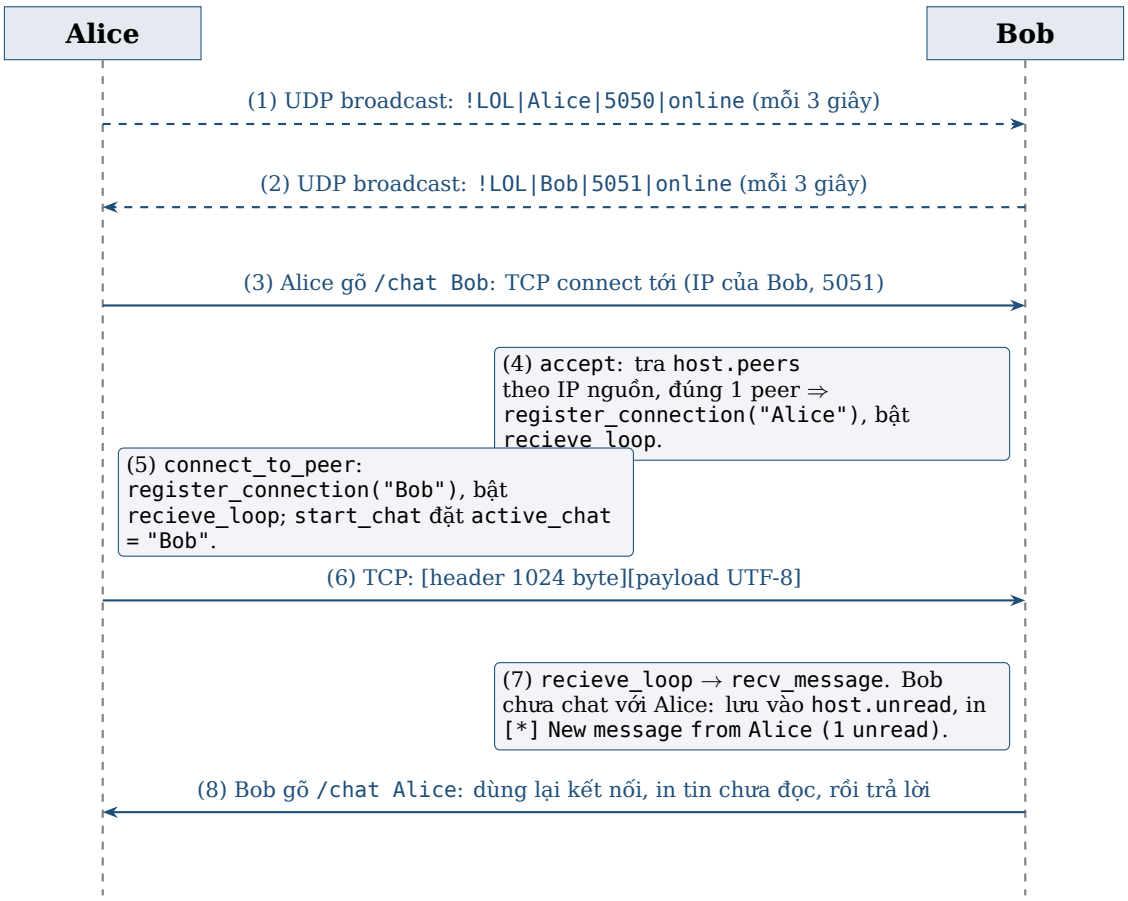
Khi nhấn Ctrl+C, `KeyboardInterrupt` được bắt ở khối `except` ngoài cùng, in `[*] Connection closed` và chương trình kết thúc; nhánh này không gọi `broadcaster.end_process()`.

Các điểm cần biết về hành vi hiện tại của main.py

- Trợ giúp ghi `/chat <name|number>`, nhưng mã gán trực tiếp `command[1]` làm tên peer, nên tham số được hiểu là tên.
- Lệnh `/send <file>` có trong trợ giúp nhưng vòng lặp chính không có nhánh xử lý nó. Lệnh không khớp nhánh nào (kể cả `/status` sai số đối số) được bỏ qua mà không báo lỗi.
- Giá trị trả về của `start_chat` không được kiểm tra: dù kết nối thất bại, chương trình vẫn vào vòng lặp chat. Khi đó `active_chat` không đổi và `send_message` trả về `False` mà không có thông báo.
- Trong vòng lặp chat, các chuỗi như `/quit` hay `/help` được coi là tin nhắn thường; chỉ `/leave` có tác dụng đặc biệt.
- Lệnh `/leave` chỉ giải quyết trong vòng lặp chat; nhập ở vòng lệnh chính sẽ bị bỏ qua.
- Khi `/quit`, `ChatManager` không được yêu cầu dừng và các socket TCP không được đóng tường minh; chúng chấm dứt cùng tiến trình.

9. Kịch bản hoạt động từ đầu đến cuối

Giả sử Alice (TCP 5050) và Bob (TCP 5051) cùng chạy chương trình trong một mạng LAN.



Hình 4: Trình tự khám phá, kết nối và gửi tin nhắn đầu tiên giữa hai peer.

1. Cả hai peer khởi động: tạo `Peer`, `Broadcaster`, `ChatManager`. Các luồng UDP và luồng `accept` bắt đầu chạy.
2. Mỗi 3 giây mỗi bên phát gói UDP; bên kia nhận, bỏ qua gói của chính mình, và ghi `host.peers[<tên>] = \{ip, port, last_seen, state\}`.
3. Alice gõ `/list` và thấy Bob; sau đó gõ `/chat Bob`. `start_chat` → `connect_to_peer`: chưa có kết nối, tra được địa chỉ của Bob, kết nối TCP, đăng ký và bật luồng nhận phía Alice.
4. Phía Bob, `accept` nhận kết nối, đối chiếu IP nguồn với `host.peers` để biết đó là Alice, đăng ký và bật luồng nhận phía Bob.
5. Alice gõ tin nhắn. `send_message` mã hóa, dựng header 1024 byte rồi `sendall`.
6. Luồng nhận của Bob đọc header, rồi payload. Vì `active_chat` của Bob chưa phải "Alice", tin được cất vào `host.unread["Alice"]` và Bob thấy thông báo số tin chưa đọc.
7. Bob gõ `/chat Alice`. `connect_to_peer` thấy đã có kết nối nên dùng lại; `active_chat` đổi thành "Alice"; các tin chưa đọc được in ra. Từ đây tin của Alice tới được in thẳng.
8. Khi một bên gõ `/leave`, `leave_chat` đặt `active_chat = None`; kết nối vẫn mở

và các tin tiếp theo từ peer đó lại được xếp vào hàng đợi chưa đọc.

A. Bảng tổng hợp toàn bộ hàm

Hàm / phương thức	Tham số	Trả về	Chức năng
peer.py			
`Peer.__init__`	`peer_name`	—	Tạo trạng thái dùng chung: tên, trạng thái, `peers`, `unread` và các khóa.
`print_help_message`	—	—	In danh sách lệnh.
broadcast.py			
`Broadcaster.__init__`	`host, bcast, tcp_port`	—	Tạo 2 socket UDP, 2 luồng daemon và `stop_flag`.
`make_udp_socket`	—	socket	Tạo socket UDP có `SO_REUSEADDR`, `SO_BROADCAST`, gắn cổng 9191.
`recieve_message`	`host`	—	Vòng lặp nhận gói UDP, lọc gói của mình, cập nhật `host.peers`.
`broadcast_message`	`host, bcast, tcp_port`	—	Vòng lặp phát gói hiện diện mỗi 3 giây.
`start_listening`	—	—	Khởi động luồng nhận UDP.
`start_broadcasting`	—	—	Khởi động luồng phát UDP.
`end_process`	—	—	Đặt `stop_flag`.
chatting.py			
`ChatManager.__init__`	`tcp_port, host`	—	Mở socket lắng nghe TCP và bật luồng `accept`.
`accept`	—	—	Chấp nhận kết nối đến, nhận diện peer theo IP, gọi `register_connection`.
`register_connection`	`peer_name, conn`	—	Đưa socket vào `connections` (bỏ socket mới nếu đã có) và bật `recieve_loop`.
`connect_to_peer`	`peer_name`	`bool`	Tái sử dụng hoặc tạo kết nối TCP chủ động tới peer.
`close_connection`	`peer_name`	—	Gỡ mục khỏi `connections` và đóng socket.
`start_chat`	`peer_name`	`bool`	Bảo đảm kết nối, đặt `active_chat`, in tin chưa đọc.
`leave_chat`	—	—	Đặt `active_chat = None`.
`recv_exact`	`conn, size`	`bytes` hoặc `None`	Đọc đúng `size` byte từ socket.

(còn tiếp ở trang sau)

Hàm / phương thức	Tham số	Trả về	Chức năng
`recv_message`	`conn`	`str` hoặc `None`	Đọc một khung (header + payload) và giải mã.
`send_message`	`peer_name, message`	`bool`	Đóng khung và gửi tin nhắn qua kết nối đã có.
`recieve_loop`	`peer_name, conn`	—	Vòng lặp đọc của một kết nối; in ngay hoặc lưu vào hàng đợi chưa đọc.
main.py			
`__main__` (khối chính)	`--name, --bcast, --tcp-port`	—	Khởi tạo các đối tượng và chạy vòng lặp lệnh.